Measure It

Segment it, measure it, prove it

AI / Computer Vision Technical Assessment

Segmentation Calibration Metric measurement 8 hours

ISSUED **29 October 2026 — 8 hours of work, submit within 24 hours**

Field	Details
Project Name	**Measure It — Segmentation and Metric Measurement**
Client	**Internal**
Developed by	**xis.ai**
Prepared by	**Abdul Moiz Qarni**
Role	AI Discipline Lead, xis.ai
Start Date	29 October 2026
End Date	24 hours from start (8 hours of work)
Status	Active
Version	2.0 (replaces the v1.0 framework)

Read this document before you start. It takes ten minutes and saves you more.

Contents

1 The Task **2**

2 Time **2**

3 What You Deliver **2**

4 Setup **3**

4.1 What You Need . . . 3

4.2 Allowed Tools . . . 4

4.3 Repository . . . 4

5 Rules **4**

5.1 Git . . . 4

5.2 What You Submit . . . 5

5.3 The Plan . . . 5

5.4 Definition of Done . . . 5

5.5 Data and Reproducibility . . . 5

6 Objectives **6**

7 The Recording **7**

7.1 What To Record . . . 7

7.2 The Four Questions . . . 7

8 How You Are Assessed **7**

9 Automatic Rejection **8**

10 Submission **8**

1 The Task

You will build a system that photographs an object and reports its real width and height in millimetres.

Pick a physical object you can hold. Then:

- Calibrate your camera so lens distortion is removed from every image you use.
- Collect and label your own images of the object.
- Train a segmentation model that finds the object's outline.
- Convert that outline from pixels to millimetres using a reference object of known size in the same photo.
- Prove how accurate the numbers are against a ruler or calliper.

This is a real industrial measurement workflow, cut down to one day. Every stage feeds the next. A good mask with a wrong scale is a wrong answer, and so is a correct scale on a distorted image.

> We care less about how high your model scores than about whether you know why your measurements are right or wrong. Anyone can run a pretrained model. We want to see you build something you can check.

2 Time

8 hours of work. Submit within 24 hours of starting, because taking photos and training a model take clock time.

Model training and dependency installs run on their own. Start them early and work on something else. That waiting time does not count towards your 8 hours.

> **You are not expected to finish the whole list in Section 3.** Very few people do. Stop where you stop, then write down what you did not reach and why.
>
> An honest, unfinished submission scores better than a rushed, broken one. A measurement system that reports a wrong number confidently is worse than one that reports nothing.

3 What You Deliver

Work down this list in order.

#	Requirement
1	**Calibrated camera.** 20 or more checkerboard or ChArUco images, calibrated with OpenCV. The camera matrix and distortion coefficients are saved to a file your code loads, and the reprojection error is reported.
2	**Your own labelled dataset.** 70 or more images of your object, taken with the calibrated camera, undistorted, and labelled with segmentation masks. The 70 / 20 / 10 train / val / test split is fixed by a committed file list.

#	Requirement
3	**A trained segmentation model.** Not an Ultralytics YOLO model and not a Roboflow model; say why you chose yours. Evaluated on the test split: mask IoU, mAP@0.5, mAP@0.5:0.95, precision and recall.
4	**One command, image to millimetres.** It takes one new image, undistorts it, segments the object, finds the reference object, and outputs a mask overlay with width (mm), height (mm) and confidence. It handles two cases cleanly: object not found, and reference not found.
5	**Accuracy against a calliper.** Measure 10 or more instances by hand and with your system. Report a per-instance table, the mean absolute error (mm) and the mean percentage error.
6	**Proof that undistortion matters.** Measure the same instances on the raw images and on the undistorted ones. Report both errors side by side. Explain the difference with your own numbers, not with theory alone.
7	**One speed or accuracy target.** You set it, you measure it, you report the result. Section 6.
8	**One condition that breaks it.** Choose one: camera tilt, distance, lighting, or the object near the edge of the frame. Measure how error changes as that condition gets worse. Say why you chose it.
9	**Knowing when not to answer.** The system refuses to report a measurement it cannot trust, for example a low-confidence mask, a partly hidden reference, or a tilted plane. Show where your threshold sits and what it costs in refused images.
10	**A decision record, written inside your Plan.** The approach you took, the approach you rejected, and what rejecting it cost.

> **Items 1 to 4 are the floor.** Nothing further is assessed until those four work. Do not start item 8 with item 5 unfinished: an accuracy claim nobody has checked is not a result.

4 Setup

Short on purpose. Installing libraries is not what we are assessing.

4.1 What You Need

- **A camera.** A phone is fine. Lock focus, zoom and resolution before you calibrate, and keep them locked for every image after. A different focus or zoom is a different camera.
- **A calibration board.** A printed checkerboard with at least 7×9 inner corners, or a ChArUco board, taped flat to something rigid. Measure a square with your calliper; do not trust the size in the file, because printers rescale.
- **A reference object.** Something rigid and flat with a known size, lying in the same plane as your object in every measurement photo. Measure it yourself, even if it has a standard size.
- **A ruler or calliper** for ground truth.
- **Your object.** Choose something you can photograph many times. Write down its approximate size and why you chose it.

4.2 Allowed Tools

- Calibration and undistortion: OpenCV.
- Labelling: CVAT, Label Studio, Roboflow's annotation tool, or similar.
- Model: any segmentation architecture except Ultralytics YOLO models and Roboflow models. Pretrained backbones are fine; the model must be trained or fine-tuned on your own images.
- AI assistants: use any you like while building. Section 7 explains how we check you understand the result.

4.3 Repository

Create a new public GitHub repository, then:

```
git checkout -b dev-test01
```

Suggested layout. Equivalent is fine; tidy is not optional.

```
calibration/      calibration images, script, saved parameters
dataset/          images, labels, split file lists
models/           training config and code (weights linked, see 5.5)
inference/        the image-to-millimetres command
measurement/      accuracy data and scripts
docs/             the three documents
README.md         setup, how to run, dataset summary
requirements.txt
```

> **You are ready** when you can load one of your own images, undistort it, and see straight edges stay straight.

5 Rules

These apply no matter how far you get.

5.1 Git

- Work on branch `dev-test01`.
- Commit as you go, in steps that show the work progressing: calibration, then data, then training, then measurement.
- Commit messages say what changed and why. Not `fix` or `wip`.
- **Commit your Plan and your test split file list first, before any code.** Git records the order and we read it.

> One large commit at the end is an automatic rejection. So is a first commit that already holds the finished pipeline.

5.2 What You Submit

Three documents under `docs/`, plus a README. A missing document is an automatic rejection, whatever your code does.

Document	What goes in it
Plan	How you intend to reach the task. Committed before any code. Section 5.3.
Objectives	The targets you set yourself and how you measured them, including calibration error, model metrics and measurement error. Section 6.
Definition of Done	How you decided the work was finished, ticked with evidence. Section 5.4.
README	Setup, how to run each stage, your hardware, and a short dataset summary: object, image count, split, labelling tool.

> Three documents, not ten. We would rather read three that are thought through than ten filled in to tick a box. Put calibration parameters, training curves and the accuracy table where they prove an objective, not in separate reports.

5.3 The Plan

One page, committed first. What you will do, in what order, and roughly how long each part should take out of your 8 hours. Include what you have already decided to skip.

If you reach item 10, close the Plan with a short decision record: the approach you took, the approach you rejected, and what rejecting it cost.

A plan that matches what happened is good. A plan that changed, with a note saying why, is better. A plan written at the end to match the result is what we are looking for, and we find it.

5.4 Definition of Done

Your own checklist, written before you start, ticked at the end with a number or a link beside each line. It is how you decide the work is finished, rather than guessing.

Something like this, but yours:

- ☐ Reprojection error reported, with the calibration image count.
- ☐ Test split committed before training started (commit link).
- ☐ Command runs on a new image and prints width and height in mm.
- ☐ Accuracy table for 10 instances, raw outputs saved.
- ☐ Target met, or missed with the reason written down.
- ☐ Everything I did not finish is listed.

> A checklist with every box ticked and no evidence beside it counts as unticked.

5.5 Data and Reproducibility

- Every image is one you took. No downloaded datasets for your object.

- The test split is chosen before training and never used for training, tuning or picking a checkpoint.
- Pin your random seed, library versions and training config. Someone else running your command should get your numbers.
- If weights are too large for Git, publish them as a release asset or link, and put the file's SHA-256 hash in the README.

6 Objectives

An objective is measurable when two people, running your method on the same setup, would agree on whether you hit it. *"The measurements should be accurate"* is not measurable. Nobody can check it.

You must write at least two:

- **MO-1: measurement accuracy.** Required. You choose the error target.
- **MO-2: your choice.** For example inference time, mask IoU, or error under the condition from item 8.

Use this format. The example uses a different problem, so you cannot copy it.

ID	Field	Entry
MO-X	What is measured	Fraction of photos of an electricity meter where every digit is read correctly.
	How	A script compares the system's output with a hand-typed answer file for each photo.
	Target	95% or more, over 60 held-out photos.
	Conditions	Phone camera, indoor light, display within 20° of face-on, same test photos every run.
	Not included	Photos where glare covers a digit, and meters with a rolling digit mid-change.

> **You pick the target number, then you justify it.** Ask what accuracy would be useful for your object, not what you think you can reach. A target you clear without doing anything tells us one thing about you. A target you miss honestly tells us something better.

Rules for any number you report:

- It comes from your machine, pasted as raw output.
- Your docs state your CPU, GPU, RAM, operating system, camera model and image resolution.
- **Timings:** measure 5 times. Report the median and the spread.
- **Errors:** show every instance in a table, then the mean, median and worst case. A mean alone hides the one measurement that was 15 mm out.

> One number on its own is not a measurement.

7 The Recording

This decides whether your submission is assessed at all.

Use any tool you like while building, including AI assistants. We do not ask and we do not mind. What we check is whether you understand what you handed in, and we check it by watching you explain it.

7.1 What To Record

One Loom, 5 minutes or less. Roughly:

Time	What you show
1 minute	Take a new photo of your object on camera, run your command on it, then measure the object with your ruler or calliper on camera.
1.5 minutes	Your own code, following that one image from file to millimetres: undistortion, inference, reference detection, conversion.
2.5 minutes	The four questions below, answered out loud.

7.2 The Four Questions

Answer all four on camera, without reading from notes.

ID	Question
K1	Which file and line turns pixels into millimetres? What has to be true about the scene for that conversion to be valid?
K2	Where in your code is undistortion applied, and how much did skipping it change your measurement error? Use your own numbers.
K3	Name one thing in your own work you would reverse, and what changing it would cost.
K4	Name one thing you did not understand, and what you did about it.

How this is checked. We watch the recording with your code open. If an answer describes something your repository does not contain, the recording fails and nothing else is assessed, however good the code is.

"I do not know, here is what I tried" never fails. A confident wrong answer always does.

8 How You Are Assessed

Three areas, in this order of importance.

Area	What we look at
What works	The parts that actually run, shown by evidence in your Definition of Done. Measurement accuracy you have proved counts for more than model metrics you have reported. You get partial credit for incomplete work you declare, and nothing for incomplete work you hide.

Area	What we look at
The three documents	Your Plan, your Objectives and your Definition of Done. Read closely, because there are only three. We look for thinking, not length.
Code presentation	Where your code sits, how it is named, how your commits read, and whether it reproduces. No dead code, no commented-out blocks, no stray files, no notebooks that only run in one order on one machine.

> Your three documents carry more weight than the code itself. The recording gates everything. A perfect pipeline you cannot explain scores nothing.

9 Automatic Rejection

Any one of these ends the assessment.

- ☐ Any of the three documents missing.
- ☐ One large commit, or a first commit holding finished code.
- ☐ Test images used for training, tuning or picking a checkpoint.
- ☐ Images of your object that you did not take yourself.
- ☐ An Ultralytics YOLO or Roboflow model used as the segmentation model.
- ☐ Recording failed.
- ☐ No recording, or a recording over 5 minutes.
- ☐ Numbers that cannot have come from the submitted work, or cannot be reproduced from it.

10 Submission

- A public GitHub repository on your own account, branch `dev-test01`.
- All three documents under `docs/` on that branch, and the README at the root.
- One pull request from `dev-test01` into the main branch of your own repository.
- Model weights in the repository or linked, with their SHA-256 hash in the README.
- Reply to the message that sent you this document with the pull request link and the Loom link, within 24 hours of starting.

Prepared by Abdul Moiz Qarni, AI / Computer Vision Department.
Issued: 29 October 2026 Version 2.0

xis.ai